

DECODING CHARACTER DEVICE

File Operations

Part—6

This article, which is part of the series on Linux device drivers, continues to cover the various concepts of character drivers and their implementation, which was dealt with in the previous two articles.

So, what was your guess on how Shweta would crack the problem? Obviously, with the help of Pugs. Wasn't it obvious? In our previous article, we saw how Shweta was puzzled by not being able to read any data, even after writing into the '/dev/mynull' character device file. Suddenly, a bell rang—not inside her head, but a real one at the door. And for sure, there was Pugs.

"How come you're here?" exclaimed Shweta.

"I read your tweet. It's cool that you cracked your first character driver all on your own. That's amazing. So, what are you up to now?" asked Pugs.

"I'll tell you, on the condition that you do not play spoilsport," replied Shweta.

Pugs smiled, "Okay, I'll only give you advice."

"And that too, only if I ask for it! I am trying to understand character device file operations," said Shweta.

Pugs perked up, saying, "I have an idea. Why don't you decode and then explain what you've understood about it?" Shweta felt that was a good idea. She *tail*'ed the *dmesg* log to observe the *printk* output from her driver. Alongside, she opened

her 'null' driver code on her console, specifically observing the device file operations *my_open*, *my_close*, *my_read*, and *my_write*.

```
static int my_open(struct inode *i, struct file *f)
{
    printk(KERN_INFO "Driver: open()\n");
    return 0;
}

static int my_close(struct inode *i, struct file *f)
{
    printk(KERN_INFO "Driver: close()\n");
    return 0;
}

static ssize_t my_read(struct file *f, char __user *buf, size_t len, loff_t *off)
{
    printk(KERN_INFO "Driver: read()\n");
    return 0;
}
```